Cloud IAM and RBAC Lab Using Auth0

Sreeram Gurumurthy Ramesh

Project Write-Up

Best Identity and Access Management Solutions

How I Got Started

I built this project because I wanted to actually understand how identity and access management works in a cloud environment, not just read about it. I had been studying IAM concepts for a while, but never really got my hands dirty with it. So I decided to set up my own lab using Auth0 and see how it all comes together.

For the scenario, I went with a K-12 school setup. The reason I picked that is that schools have a pretty natural mix of user types: students, teachers, admin staff, helpdesk, and IT admins, and each group clearly needs different access. It made it easy to think about who should be allowed to do what without overcomplicating things.

What I Was Trying to Solve

When I was studying for my security certifications, one thing that kept coming up was how many breaches happen because of identity issues, weak passwords, people having way more access than they need, no MFA, and no logs to check when something goes wrong. I wanted to build something that actually addresses those problems, even if it was just a lab.

The specific things I wanted to fix were:

- People logging in with just a password and nothing else.
- Users have access to things they do not actually need for their job.
- No way to check what happened if something goes wrong.

• Admins have too much access with no separation between helpdesk and IT admin tasks.

Setting It Up

I used Auth0 for everything. It is a cloud platform that handles authentication and authorisation, and you can configure a lot of things through the dashboard without writing code. That made it a good fit for a lab where I wanted to focus on the IAM concepts rather than getting stuck on infrastructure.

The first thing I did was create the user accounts. I made 8 test users to represent each type of person in the school. Then I created the roles and linked each user to the right one.

Username	Role
student01	K12_Student
student02	K12_Student
teacher01	K12_Teacher
teacher02	K12_Teacher
registrar01	K12_Admin_Staff
principal01	K12_Admin_Staff
helpdesk01	K12_IT_Helpdesk
admin.lab	K12_IT_Admin

After the users and roles were set up, I created two sample applications inside Auth0 to represent school apps, and then I defined API permissions for those apps. The permissions are basically what each role is allowed to do inside those applications.

The Access Model

This part took me a bit of thinking to get right. The idea is that permissions are assigned to roles, and roles are assigned to users. You never assign permissions directly to a user. That way, if someone changes jobs or leaves, you just change their role, and everything updates automatically.

Here are the four permissions I created and what they mean:

Permission	What it allows
read:learning_portal	View the student learning portal
read:gradebook	View grade information
manage:users	Create or update user accounts
read:logs	View system activity logs

And here is how I mapped those permissions to each role:

Role	Permissions
K12_Student	read:learning_portal
K12_Teacher	read:learning_portal, read:gradebook
K12_Admin_Staff	read:gradebook
K12_IT_Helpdesk	manage:users
K12_IT_Admin	manage:users, read:logs

One thing I paid attention to was keeping the helpdesk and IT admin separate. Helpdesk can manage users, but they cannot see logs. IT admin can do both. It might seem like a small thing, but it matters. If a helpdesk account gets compromised, the attacker cannot also read the logs and **cover their tracks.**

Security Controls I Put In Place

Beyond just the roles and permissions, I also configured a few other controls to make the setup more realistic.

What I was worried about	What I did	Why it helps
Someone is stealing a password.	Turned on MFA for all Users.	Even with the password, you still need the second factor to get in.
Users having too much access.	Used RBAC with specific roles.	Each person can only access what their role allows.
Apps not checking permissions.	Set up API permissions in Auth0.	Apps verify the permission scope before allowing access.
Not knowing what happened.	Enabled Auth0 monitoring logs.	Every login, Failure and change gets recorded.
Admin accounts being too powerful	Split IT admin and helpdesk into separate roles.	Limits how much damage one compromised account can do.

What I Found in the Logs

Once everything was set up, I went through the Auth0 logs to see what was being captured. The user creation events and logins showed up fine. But I also noticed some failed notification events. Basically, Auth0 trying to send verification emails to the test accounts and failing because I used

fake email addresses.

At first I was a bit confused by those errors, but then I realized it actually makes sense. The lab was never going to have real inboxes, so the emails were always going to bounce. What I found interesting is how quickly you can spot that kind of thing in the logs. In a real environment, those same failed notifications might mean something is misconfigured and worth investigating.

What I Took Away from This

Honestly, this project taught me more than just reading about IAM ever did. A few things that stuck with me:

• RBAC feels simple on paper, but you have to think carefully about it. It is easy to accidentally give a role too much access if you are not deliberate about it.

• MFA is straightforward to set up in Auth0. There is really no excuse not to have it enabled, even in a lab.

• Separating roles like helpdesk and IT admin is not just a nice-to-have. It is basic least privilege, and it limits the blast radius if something goes wrong.

• Logs told me things I did not expect. The failed email notifications were not a problem I planned for, but they showed up clearly and were easy to understand.

• Doing this in a cloud platform made it fast. No servers to set up, no config files to mess with, just the IAM concepts themselves.